

TEMA 66. ENTORNO DE DESARROLLO MICROSOFT .NET

Actualizado a 13/04/2023

1. INTRODUCCIÓN

La plataforma .NET es un entorno de desarrollo de software desarrollado por Microsoft que se lanzó en el año 2002. .NET se diseñó como una plataforma unificada para desarrollar aplicaciones de software en diferentes lenguajes de programación, incluyendo C#, Visual Basic y F#.

Microsoft lanzó su plataforma .NET, que competía directamente con la JVM de Oracle. Ambas plataformas ofrecían entornos de tiempo de ejecución basados en máquinas virtuales y código intermedio para desarrollar aplicaciones que podrían ejecutarse en hardware y versiones de OS distintas.

Uno de los mayores puntos de debate entre .NET y JVM fue el lenguaje de programación preferido de cada plataforma. La JVM se centraba en el lenguaje de programación Java, mientras que .NET se enfocaba en C# y Visual Basic (evento conocido como “la guerra de los lenguajes”).

Además, había diferencias en la arquitectura de cada plataforma. La JVM utilizaba un modelo de máquina virtual que se ejecutaba en la parte superior del sistema operativo, mientras que .NET se integraba más estrechamente con el sistema operativo Windows. Esto significaba que los desarrolladores tenían que elegir entre la portabilidad de la JVM y la integración más profunda con el sistema operativo de .NET.

A pesar de la competencia, ambas plataformas continuaron evolucionando y mejorando con el tiempo. En muchos casos copiando elementos de la competencia y evolucionando en paralelo. Sin embargo, la versión oficial .NET seguía limitada a entornos Windows, por lo que se desarrollaron variantes del entorno .NET independientes e incompatibles, dando lugar a un ecosistema limitado y fragmentado.

En 2016, Microsoft lanza la plataforma .NET Core, con los objetivos de unificar los entornos en una única plataforma, desarrollar aplicaciones modernas y modulares, crear una comunidad activa haciendo la plataforma software abierto y expandirse a otros sistemas operativos (como Linux o macOS) y arquitecturas (como arm64).

Actualmente se está eliminando progresivamente la plataforma .NET Framework y .NET Core se ha renombrado a .NET como única plataforma unificada. La última versión LTS es la 6, con la versión 7 ya publicada y la versión 8 (LTS) propuesta para finales de 2023.

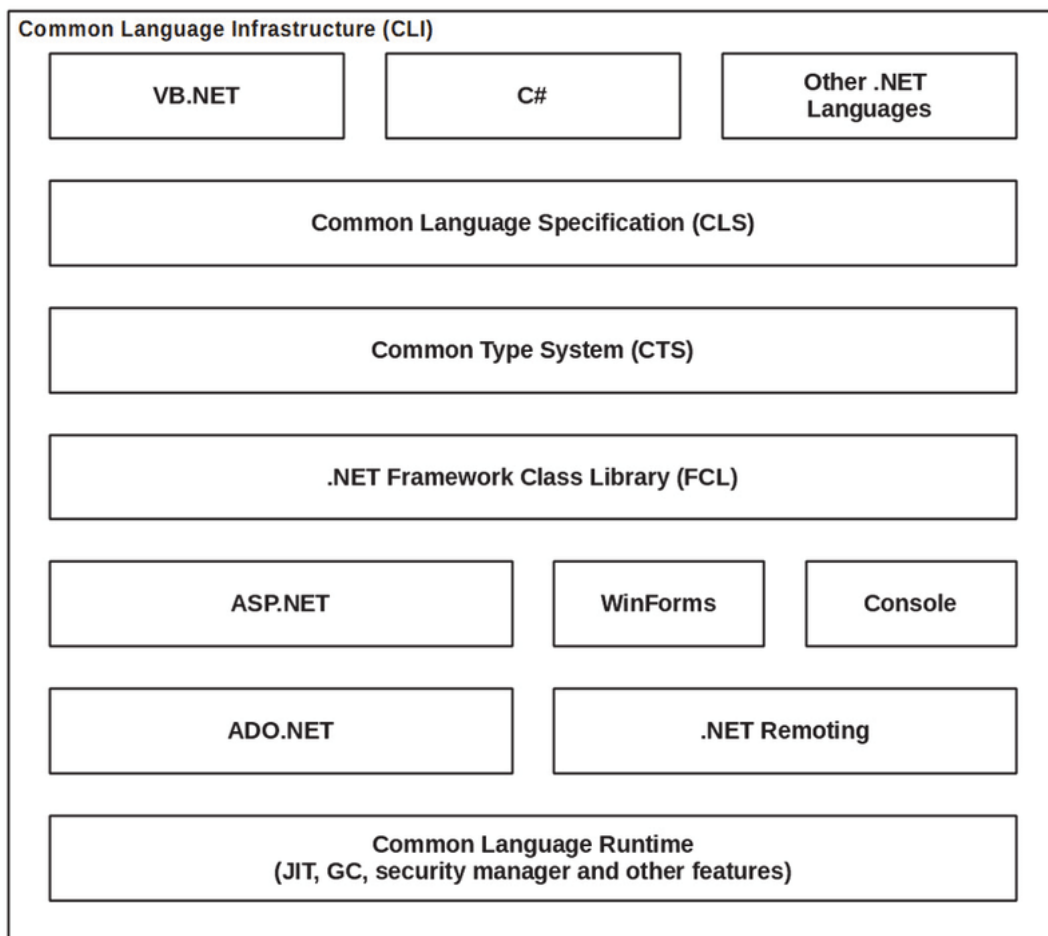
2. ECOSISTEMA .NET

Los principales componentes del ecosistema .NET son los siguiente:

- .NET Standard (especificación formal de las APIs y librerías disponibles)
- .Net Implementations (.NET Framework, .NET (Core, Mono, Universal Windows Platform)
- .NET Runtimes (CLR, CoreCLR, Mono Runtime, .Net native for UWP).
- .NET tooling, CLI and Common Language Infrastructure (compilador y otras herramientas).
- Bibliotecas de clases y tipos (BCL, FCL, CTS)
- Servidores .NET Enterprise Servers e Internet Information Service (IIS)
- Lenguajes de programación (C#, F#, VB.NET, Otros)
- Entornos de desarrollo (Visual Studio, Visual Studio Code/VSCodium, Monodevelop, JetBrains Rider)
- Gestores de paquetes (NuGet)
- Frameworks de desarrollo (ASP.NET, Xamarin/MAUI)

3. .NET FRAMEWORK

.NET Framework es un conjunto de tecnologías de software desarrollado por Microsoft en 2002 que proporciona **una plataforma para crear, ejecutar y desplegar aplicaciones y servicios en entornos Windows**. La plataforma .NET incluye un entorno en tiempo de ejecución llamado Common Language Runtime (CLR), una biblioteca de clases de .NET Framework (FCL), un conjunto de compiladores para varios lenguajes de programación y otros componentes y herramientas para la creación de aplicaciones y servicios para Windows, Web, móviles y otros dispositivos.



https://www.researchgate.net/publication/325534947_Performance_Comparison_of_Java_EE_and_ASP.NET_Core_Technologies_for_Web_API_Development

.NET Framework es una implementación de la especificación de **Common Language Infrastructure (CLI)**, que define una plataforma de tiempo de ejecución para lenguajes de programación de alto nivel. Esta especificación está definida en el estándar **ECMA-335 (ISO/IEC 23271) del año 2001/2002**.

Actualmente se encuentra en su **versión 4.8.1** con soporte de actualizaciones extendido sin embargo no se recomienda utilizar este framework para nuevos proyectos dado que los planes futuros del creador de la plataforma (Microsoft) pasan por dejar este framework obsoleto (en el medio-largo plazo) y dejar una única plataforma de desarrollo, .NET.

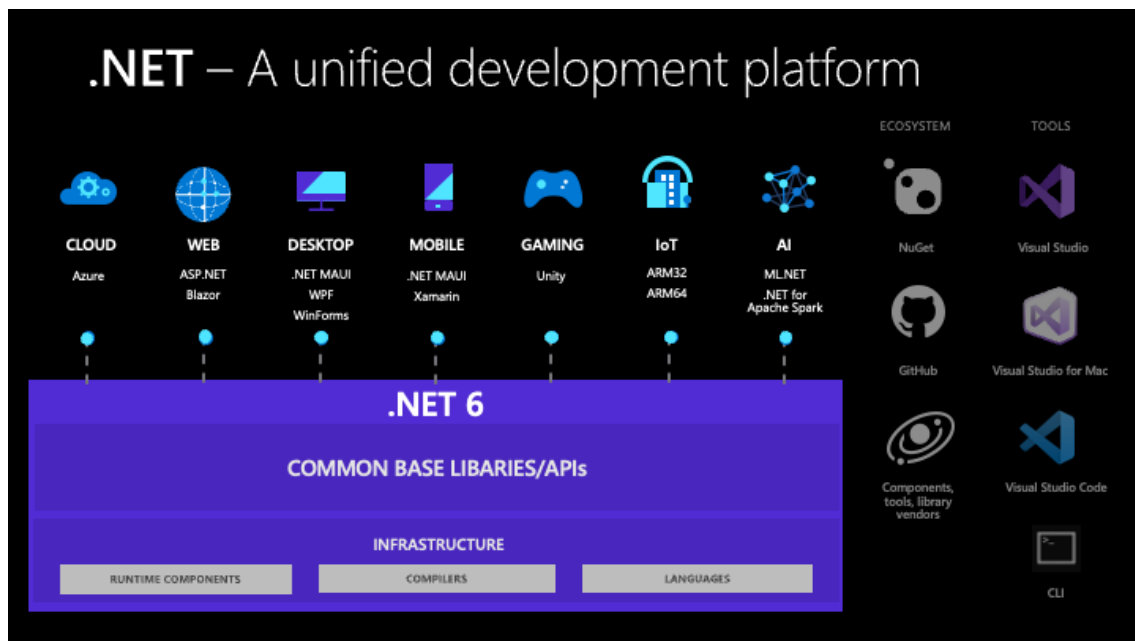
4. .NET (CORE)

.NET Core es una plataforma de desarrollo de software gratuita, de código abierto y multiplataforma, creada por Microsoft en 2016. Esta plataforma tiene como objetivo permitir a los desarrolladores crear aplicaciones modernas y escalables para una amplia gama de dispositivos y sistemas operativos.

Inicialmente, .NET Core fue diseñado como una versión más liviana y modular del framework .NET Framework de Microsoft, que se centraba en Windows y estaba limitado a aplicaciones de escritorio. Con .NET Core, los desarrolladores podrían crear aplicaciones para Windows, macOS y Linux, así como para dispositivos móviles y la nube.

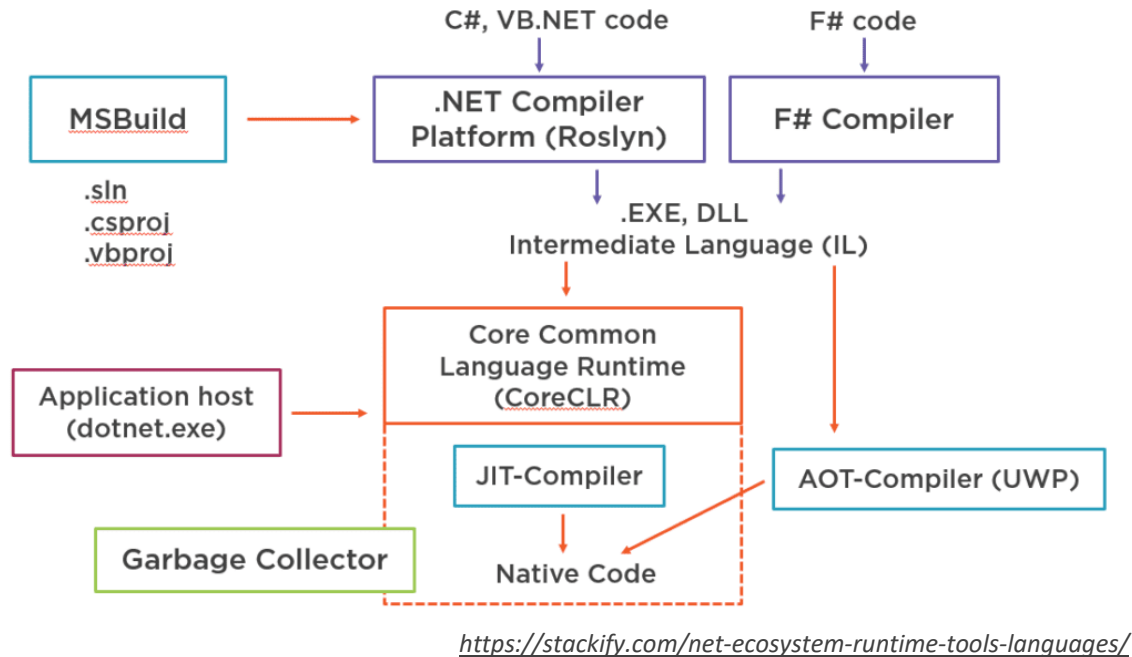
En 2019, Microsoft anunció que .NET Core sería renombrado como .NET y se uniría a la plataforma .NET Framework y Xamarin, una plataforma de desarrollo móvil, bajo la marca .NET 5. El objetivo de esta unificación fue ofrecer a los desarrolladores una única plataforma de desarrollo para crear aplicaciones para cualquier dispositivo y cualquier sistema operativo.

En la actualidad es la plataforma preferente de desarrollo y con soporte y desarrollo activo. A principios de 2023 **la versión LTS es .NET 6, con .NET 8 programada para finales de 2023.** Las versiones impares de la plataforma no serán de soporte extendido y se espera que se publique al menos una versión nueva por años con un listado de funcionalidades pendientes disponible en GitHub.

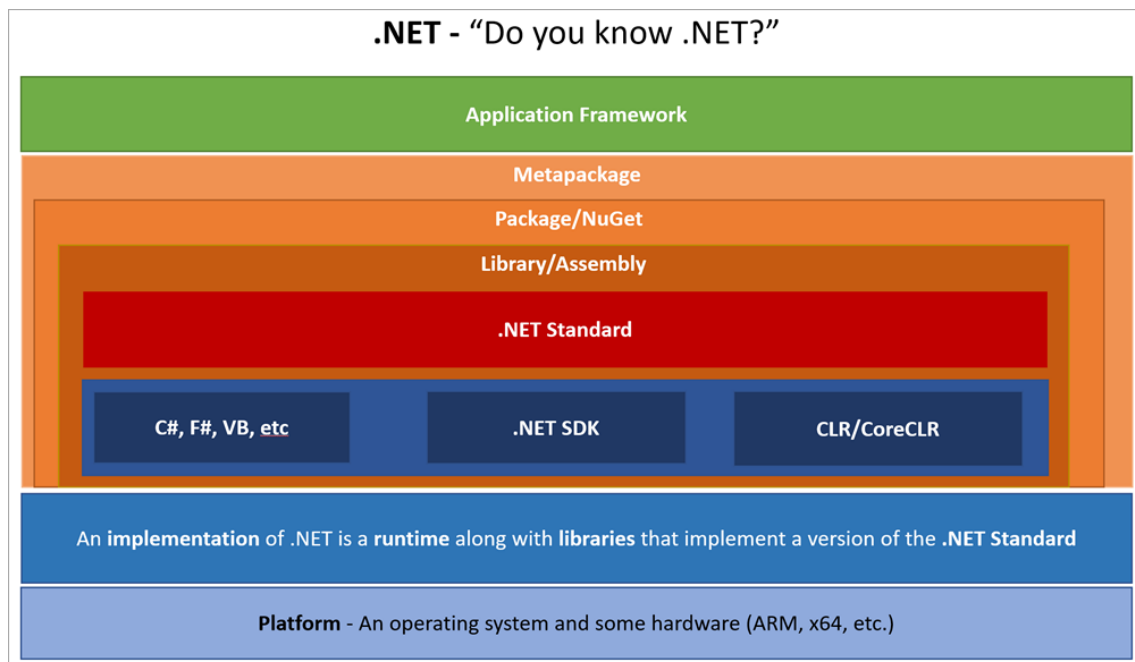


<https://devblogs.microsoft.com/dotnet/announcing-net-6/>

Con la llegada de .NET Core se han realizado varios cambios a nivel de arquitectura. Por un lado, el núcleo de ejecución de .NET, el CLR, se ha reescrito por completo para soportar múltiples plataformas y se ha publicado como código abierto. **La librería del framework (FCL) se ha sustituido por .NET Standard y la Base Class Library (BCL) de .NET. FCL es un superset de BCL** que no incluye llamadas antiguas obsoletas o específicas de Windows que no se pueden migrar. Dado que se busca una plataforma unificada para desarrollo multiplataforma y multientorno, se ha desarrollado un BCL nuevo. Se mantiene el uso del sistema común de tipos (CTS) y de definición de lenguajes (CLS) para dar soporte a los diferentes lenguajes de la plataforma.



5. .NET STANDARD



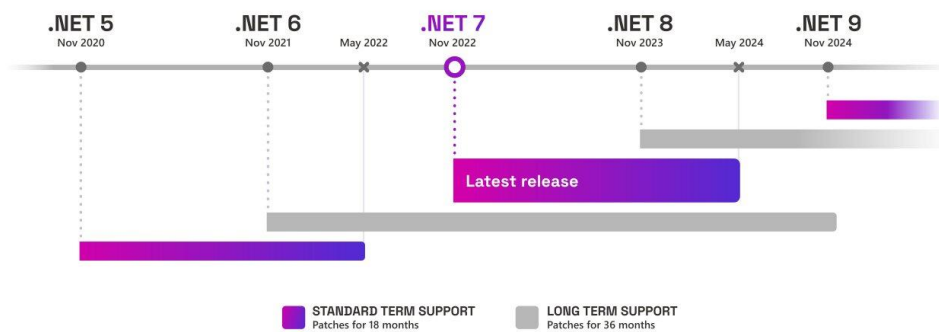
.NET Standard es una especificación de biblioteca de clases que se introdujo en 2016 junto con .NET Core 1.0 y se actualizó posteriormente con cada versión importante de .NET Core. El objetivo principal de .NET Standard es proporcionar una API común que se puede utilizar en todas las implementaciones de .NET, incluido .NET Core, .NET Framework y Xamarin. Esto significa que los desarrolladores pueden crear bibliotecas que funcionen en múltiples plataformas de .NET sin necesidad de reescribir el código para cada plataforma. **La última versión es .NET Standard 2.1 que soporta desde .NET Core 3.1** así como las últimas versiones de Mono, Xamarin y Unity 2021.2.

.NET Standard define un conjunto de API comunes que se espera que todas las implementaciones de .NET admitan. Esta especificación incluye una serie de bibliotecas de clases de uso común, como System, System.IO y System.Net, entre otras. Además, .NET Standard proporciona una serie de especificaciones de versiones para permitir que los desarrolladores sepan qué API se admiten en cada versión y cómo las versiones anteriores se relacionan con las versiones más nuevas.

Cabe destacar que .NET Standard es una especificación y que **cada implementación** (Mono, .NET Framework, .NET) tienen un núcleo de ejecución (CLR) distinto, pero si se ajustan a la misma versión de .NET Standard, **deben tener disponibles las mismas APIs y librerías disponibles, pero esto genera problemas en librerías de dispositivos móviles o ARM.**

Por ello, **a partir de .NET 6 se frena el desarrollo de .NET Standard, la versión 2.1 es la última versión y no se recomienda su uso en proyectos nuevos fuera de razones de compatibilidad.** Con la unificación de la plataforma se busca un único punto de referencia para móviles, ARM, Mono y Unity que será .NET (Core).

6. NOVEDADES DE .NET



<https://dotnet.microsoft.com/es-es/platform/support/policy/dotnet-core>

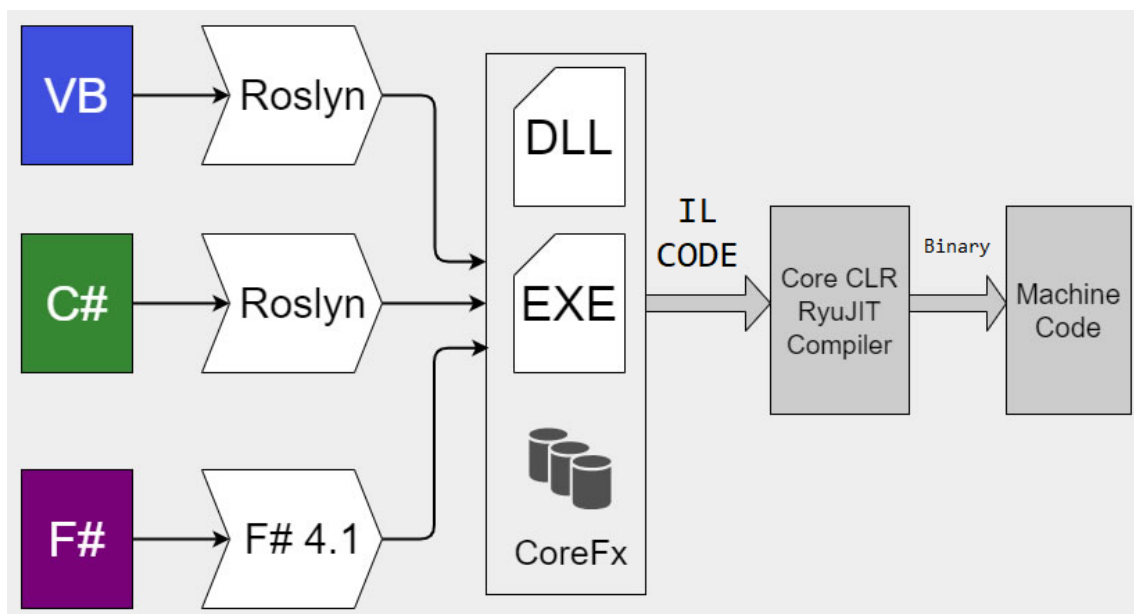
- **.NET Framework**
 - **Framework legacy**
 - LTS
 - 4.8.0
 - 4.8.1
- **.NET Standard**
 - **Especificación formal de librerías y APIs soportadas, importante para compatibilidad .NET FW**
 - Última versión
 - 2.1
- **.NET CORE 1.0**
 - **Lanzamiento en 2016**
 - Solo disponible en entornos web
- **.NET CORE 2.2**
 - Lanzamiento en 2018
 - Versión mejorada y estable
 - Inicio de entornos móviles
 - Visual Studio 2017

- .NET Core 3.1
 - Lanzamiento en 2019
 - Mejoras de rendimiento y flexibilidad
 - Inicio de aplicaciones de escritorio y mejoras en móviles
 - Visual Studio 2019
- .NET 5
 - Lanzamiento en 2020
 - Unificación de la plataforma de desarrollo y cambio de nombre
 - Blazor
 - C# 9
- **.NET 6**
 - Lanzamiento en 2021
 - **Actual versión LTS**
 - Minimal API, recarga de código, HTTP/3, WindowsForms
 - MAUI
 - C# 10
 - Visual Studio 2022
- .NET 8
 - Futura versión LTS

7. ENTORNO DE EJECUCIÓN

El ecosistema .NET, tanto .NET Framework como .NET (Core), utilizan un modelo de ejecución con máquina virtual y código intermedio similar al que se utiliza en Java. A continuación, se detallan **los elementos que componen el motor de ejecución de .NET: el CLR, el código IL, los compiladores.**

Los entornos de ejecución de .NET y .NET Framework se han desarrollado de acuerdo con el estándar CLI (Common Language Infrastructure) **CLI ECMA-335 (ISO/IEC 23271).**



<https://dotnet-talk.com/images/blog/dotnet-core-compile-diagram.jpg>

7.1 CLR/CORECLR

CLR (Common Language Runtime) es un componente central del marco de trabajo .NET de Microsoft que proporciona una capa de abstracción entre el código fuente del programa y el sistema operativo en el que se ejecuta. **CLR funciona como una máquina virtual que permite la ejecución de código escrito en diferentes lenguajes de programación y proporciona una serie de servicios de tiempo de ejecución,** como la administración de memoria, recolección de basura, seguridad, reflectividad, tipos genéricos...

CLR trabaja en conjunto con el compilador de .NET para compilar el código fuente en un lenguaje común llamado Common Intermediate Language (CIL), también conocido como código IL (Intermediate Language). Este código IL se almacena en archivos con extensión ".dll" o ".exe". Durante la ejecución del programa, CLR toma el código IL y lo compila en código de máquina nativo que puede ser ejecutado por la CPU del sistema utilizando una compilación JIT (Just-in-time) que genera el código en tiempo real a medida que el código se ejecuta. En algunos casos es posible realizar una compilación AOT (Ahead-of-time) como es el caso de la plataforma UWP o como configuración de .NET 6.

CLR es la versión original propietaria que se creó con .NET Framework y solo funciona en la plataforma Windows. CLR admite además un modelo de seguridad denominado seguridad de acceso del código o CAS (Code Access Security) para el código administrado (si no es administrado, CLR no participa en la ejecución). **CoreCLR es una versión simplificada y optimizada de CLR que fue desarrollada para ser utilizada en múltiples plataformas, como Linux y macOS.** CoreCLR es una versión modular y de código abierto de CLR que está diseñada para ser ligera y rápida en su ejecución que no admite CAS.

Los entornos de ejecución implementan su propia versión de CLR o CoreCLR:

- Mono (Implementación de código abierto de .NET Framework antes de que existiera .NET Core)
- .NET y .NET Framework
- UWP
- Xamarin
- Unity

7.2 CÓDIGO INTERMEDIO – IL

IL (Intermediate Language) es un lenguaje de nivel intermedio usado en el ecosistema .NET similar al bytecode de Java para la JVM. El compilador de .NET convierte el código fuente en código IL. Este código es un conjunto de instrucciones de bajo nivel que se asemejan a un lenguaje de ensamblador, pero es independiente de la plataforma. Este código IL puede ser compilado en código de máquina específico de la plataforma en la que se está ejecutando, lo que permite que los programas escritos en diferentes lenguajes se ejecuten en cualquier plataforma que tenga un compilador de .NET disponible.

El IL se ha extendido para soportar más acciones y flexibilidad con las nuevas plataformas, dependiendo de la fuente de información, puede tener otros nombres, pero se refiere al mismo conjunto de instrucciones de nivel intermedio del ecosistema .NET.

- **MSIL: Microsoft Intermediate Language es el nombre original del lenguaje** que actualmente se encuentra en desuso. Generalmente se utiliza para referirse a las versiones de .NET Framework.
- **IL: Intermediate Language es el nombre técnico y más común de referirse al lenguaje, especialmente en .NET Core**
- **CIL: Common Intermediate Language es el nombre formal de la especificación técnica del lenguaje**

7.3 COMPILADORES Y ROSLYN

Los compiladores de .NET se utilizan para convertir el código fuente de los programas escritos en lenguajes de programación de .NET en código IL. Cada lenguaje tiene su propio compilador tanto de los lenguajes oficiales de .NET (compilador de F# o el de VB.NET) como de implementaciones de comunidad.

Con la llegada de .NET Core se rediseñó el sistema de compilación con el nombre de Roslyn y se publicó como código abierto. Roslyn es un conjunto de herramientas y bibliotecas de compilación de código abierto de Microsoft, que se utilizan para crear compiladores y analizadores de código para C# y VB.NET.

Roslyn opera como un *Compiler-as-a-Service*, ya que también proporciona un modelo de objetos para el código C# y Visual Basic, lo que permite que los programas escritos en estos lenguajes sean analizados y manipulados de manera más avanzada que lo que se podía hacer con compiladores anteriores. Esta plataforma permite realizar un análisis sintáctico y semántico del código, refactorizar y aplicar reglas de estilos o ejecutarse bajo demanda en el código para realizar tareas de reflectividad, generación de código o análisis de árboles sintácticos.

7.4 ENSAMBLADOS

En .NET, un ensamblado es la unidad fundamental de distribución, versión y seguridad de un componente de software. Un ensamblado es un archivo que contiene el código ejecutable (IL), los recursos necesarios, metadatos y un manifiesto obligatorio con los datos del ensamblado. **Ya sean ficheros “.dll” o “.exe”, este último se llama también Portable Executable (PE)**.

Cada ensamblado en .NET tiene un nombre único, una versión asociada y cada ejecutable se ejecuta en su propio de aplicación (AppDomain) o en varios (únicamente en .NET Framework). Los ensamblados también proporcionan un mecanismo para especificar dependencias en otros componentes de software.

La principal diferencia entre los ensamblados de .NET Framework y .NET (Core) radica en las librerías utilizadas y su origen (como es el caso de `System.String` que procede en .NET de `System.Runtime` y de `mscorlib.dll` en .NET Framework). Aunque sí apuntan a la misma especificación de .NET Standard se pueden utilizar entre entornos soportados.

8. CONJUNTOS DE LIBRERÍAS Y HERRAMIENTAS

Otra parte fundamental del ecosistema .NET son las librerías, APIs y herramientas disponibles. .NET Standard 2.1 dispone de 37.118 APIs que se pueden utilizar a través de todos los lenguajes y entornos soportados. .NET Framework dispone además de varias herramientas de consola (algunas con GUI) como `Mage.exe` (Manifest Generation and Editing Tool) para gestionar de manifiestos de aplicación, `certmgr.exe` para la gestión de certificados o `sn.exe/SignTool.exe` para la firma y verificación de ensamblados.

Con .NET (Core) se crea una CLI (`dotnet.exe`) interactiva para gestionar proyectos, desde su creación vía plantillas hasta la compilación, ejecución y despliegue. Además, esta herramienta permite añadir plugins como el de Entity Framework (`dotnet-ef`) o de recarga automática de código (`dotnet-watch`). Visual Studio también incluye sus propias herramientas como `MSBuild.exe` que sirve para compilar, probar y ejecutar proyectos de .NET Framework y .NET (Core).

8.1 FCL y BCL

BCL (Base Class Library) es la biblioteca de clases base de .NET y se compone de un conjunto de clases, interfaces y tipos que proporcionan la funcionalidad fundamental para cualquier aplicación .NET. Por ejemplo, BCL proporciona clases básicas como Object, String, Array, Math, etc., que son esenciales.

FCL (Framework Class Library) es una biblioteca más amplia que se construye sobre BCL y proporciona clases adicionales y funcionalidades avanzadas para una variedad de escenarios de aplicación, como aplicaciones web, aplicaciones de escritorio, aplicaciones móviles, servicios y más. FCL incluye una variedad de bibliotecas de clases, como WINAPI, ADO.NET, Windows Forms, WPF, entre otras.

FCL es el conjunto utilizado por .NET Framework que incluye muchas librerías exclusivas de Windows, por otro lado, BCL se potencia con la llegada de .NET Core y es la que se utiliza en .NET junto con otras librerías más complejas que se han añadido progresivamente al BCL original.

Tanto BCL como FCL implementan las APIs de .NET Standard, lo que significa que cualquier biblioteca que se construya sobre FCL o BCL será compatible con cualquier implementación .NET que admita la versión de .NET Standard que se está utilizando.

9. LENGUAJES DE PROGRAMACIÓN

.NET al ser un ecosistema de código intermedio, puede soportar cualquier lenguaje, además de los oficiales, que tenga un compilador que genere código IL y que se ajuste al CLS y CTS.

9.1 CLS y CTS

Common Type System (CTS): Es una especificación que define cómo los tipos de datos se definen, usan y administran en .NET. Define reglas para la creación de tipos, la asignación de memoria, la interoperabilidad entre lenguajes y otros aspectos relacionados con el manejo de tipos en .NET.

Common Language Specification (CLS): Es una especificación que define un conjunto de reglas que los lenguajes de programación .NET deben seguir para garantizar la interoperabilidad entre lenguajes y la portabilidad del código entre diferentes plataformas y compiladores. Define un subconjunto del CTS y establece reglas para las estructuras, nombres, tipos y otros elementos que los lenguajes .NET deben admitir para que el código sea compatible con otros lenguajes .NET. Permite crear una infraestructura común de lenguajes.

9.2 C#

C# (pronunciado C Sharp) es un lenguaje de programación de alto nivel de propósito general que admite múltiples paradigmas. C# se caracteriza por admitir tipado estático, tipado fuerte, imperativo, declarativo, funcional, genérico, estar orientado a objetos (basado en clases) y orientado a componentes.

El lenguaje de programación C# fue diseñado por Anders Hejlsberg de Microsoft en el año 2000 inspirándose en C/C++, Delphi y Java entre otros. Posteriormente **fue aprobado como estándar internacional por Ecma (ECMA-334) en 2002 e ISO/IEC (ISO/IEC 23270) en 2003.**

Es el lenguaje principal de desarrollo del ecosistema .NET y uno de los principales competidores de Java. **A fecha de noviembre de 2022, la versión estable más reciente del lenguaje es C# 11.0,** que se lanzó en 2022 en .NET 7.0. En .NET Framework dispone de compilador propio y en .NET (Core) utiliza Roslyn.

9.3 VISUAL BASIC .NET

Visual Basic .NET (VB.NET), es un lenguaje de programación de múltiples paradigmas y orientado a objetos. Microsoft lanzó VB.NET en 2002 como sucesor de su lenguaje original Visual Basic, cuya última versión fue Visual Basic 6.0.

Su introducción resultó muy controvertida, ya que debido a cambios significativos en el lenguaje VB.NET no es retro compatible con Visual Basic, pero el manejo de las instrucciones es similar a versiones anteriores de Visual Basic.

Microsoft actualizó su estrategia de lenguaje VB el 6 de febrero de 2023, declarando que VB es ahora un lenguaje estable y que Microsoft lo seguirá manteniendo. La última versión es la 16.9 publicada con el Visual Studio 2019. En .NET Framework dispone de compilador propio y en .NET (Core) utiliza Roslyn.

9.4 F#

F# (pronunciado F Sharp) es un lenguaje de programación de múltiples paradigmas, de tipado fuerte y con enfoque funcional. Engloba métodos de programación funcional, imperativa y orientada a objetos. Generalmente usado como lenguaje de propósito general dentro de .NET, pero también puede generar código JavaScript y para unidades de procesamiento gráfico (GPU).

F# es desarrollado por la F# Software Foundation, Microsoft y colaboradores abiertos. F# es un miembro de la familia de lenguajes ML y se originó como una implementación del Framework .NET de un núcleo del lenguaje de programación OCaml. También ha sido influenciado por C#, Python, Haskell, Scala y Erlang.

A partir de noviembre de 2022, la versión estable más reciente del lenguaje es F# 7.0, que se lanzó en 2022 en .NET 7.0. A diferencia de C# y VB.NET utiliza un compilador propio tanto para .NET Framework como para .NET (Core). F# se ha vuelto muy popular en el ámbito de la ciencia de datos, el aprendizaje automático y la programación financiera dado su enfoque funcional y de código seguro.

9.5 OTROS LENGUAJES

Algunos lenguajes que se utilizan dentro de .NET son los siguientes junto con su correspondiente implementación de compilador a IL.

<u>.NET Framework (+100)</u>	<u>.NET (Core)</u>
• Python: IronPython • PHP: PeachPie • Java: J# • C++/CLI • Object-Pascal: Oxygene • JScript • ...	• Python: Pyjion • PHP: PeachPie • C++/CLI • Object/Pascal: Oxygene

10. NUGET

NuGet es un gestor de paquetes para la plataforma .NET de código abierto que permite buscar, instalar y administrar bibliotecas de terceros y otros componentes de software en sus proyectos. NuGet funciona como un repositorio centralizado de paquetes que los desarrolladores pueden acceder y utilizar para agregar funcionalidades a sus aplicaciones.

Utiliza ficheros zip con la extensión “.nupack” o “.nupkg” que contiene un manifiesto y los ensamblados necesarios. Dispone de una aplicación de consola y está totalmente integrado con IDEs, CLIs, herramientas de despliegue como CakeBuild y entornos de compilación automático. Existen además alternativas como Paket o MyGet.

11. ARQUITECTURA FÍSICA

Las aplicaciones del ecosistema .NET pueden desplegarse utilizando diversos servicios:

- **Servidor Integrado (Kestrel):** Kestrel es un servidor web de código abierto que se ejecuta en el framework .NET (Core). Es rápido, escalable y multiplataforma, lo que lo hace ideal para alojar aplicaciones web de alto rendimiento. Kestrel es el servidor web predeterminado para las aplicaciones .NET Core. Alternativa a Http.Sys en Windows
- **Internet Information Services (IIS):** Es un servidor web de Microsoft integrado en Windows Server y utilizado para alojar y servir sitios web. IIS es compatible con múltiples lenguajes de programación, incluyendo .NET Framework. De hecho, IIS se integra de forma nativa con .NET Framework y proporciona un entorno de alojamiento seguro y escalable para aplicaciones web desarrolladas en .NET Framework y .NET (Core)
- **Servidores web:** Las aplicaciones del ecosistema .NET se pueden desplegar en otros servidores web como NGINX o Apache mediante plugins que utilicen entornos de ejecución como es el caso de mod_mono en Apache.
- **Contenedores:** Microsoft proporciona numerosas imágenes en repositorios de contenedores como DockerHub para desplegar aplicaciones de .NET. En el caso de .NET Framework no son compatibles con contenedores Linux (al ser una plataforma de Windows exclusivamente).
- **Proveedores cloud:** Varios proveedores cloud proporcionan servicios de despliegue de aplicaciones escritas en .NET como Azure (vía contenedores, funciones o webapps), AWS o GCP. También se pueden desplegar en proveedores que soporten Docker utilizando las imágenes de .NET (Core), aunque no tantos soportan .NET Framework en esta modalidad.

Además de los servidores web, existen otros servicios empresariales para desplegar aplicaciones web:

- **BizTalk Server:** Es un servidor empresarial de integración que permite a las empresas automatizar procesos empresariales complejos y conectar sistemas dispares. BizTalk Server utiliza adaptadores predefinidos para integrarse con una amplia variedad de sistemas empresariales, incluyendo SAP, Oracle y Salesforce.

12. ENTORNOS DE DESARROLLO

Existen diversos IDEs (entorno de desarrollo integrado) que soportan desarrollo en .NET. El IDE principal de desarrollo es Visual Studio, creado por Microsoft en 1997. Es un IDE propietario y de pago (aunque existe una versión de comunidad para individuos y proyectos de software abierto) multilenguaje disponible en Windows y macOS. Admite numerosos lenguajes como JavaScript, Python, PHP, C++ y por supuesto C#, F# y VB.NET. Cada nueva versión de Visual Studio suele ir acompañada de una nueva versión de .NET (Core) y/o .NET Framework, las cuales no siempre son compatibles en versiones anteriores. La versión actual es Visual Studio 2022 que introdujo .NET 6 y .NET Framework 4.8/4.8.1.

Visual Studio no es el único entorno de desarrollo, además existe **Visual Studio Code que es un editor de código fuente gratuito y de código abierto** desarrollado por Microsoft en 2015. Disponible en Windows, Linux y macOS, internamente es una aplicación web publicada con Electron y que utiliza el motor Monaco de edición de código. Proporciona mucha flexibilidad mediante plugins y un gran número de herramientas y personalizaciones, como por ejemplo integración con la plataforma .NET. **Existe una versión paralela sin intervención de Microsoft llamada VSCodium**.

Otros IDEs que soportan desarrollo en .NET son JetBrains Rider o Monodevelop.

Además, existe la posibilidad de desarrollar en .NET (Core) de forma interactiva en la web utilizando .NET Interactive. Esta tecnología utiliza el concepto de “Notebooks”, similar a Jupyter Notebooks, muy utilizado en ciencia de datos con Python, que permiten escribir código en .NET como si fueran notas ejecutables que favorecen la colaboración en tiempo real y la experimentación.

13. DESARROLLO WEB – ASP.NET

El desarrollo de aplicaciones web es uno de los puntos fuertes del ecosistema .NET y existe una larga trayectoria de tecnologías y frameworks.

13.1 ACTIVE SERVER PAGES – ASP

En 1996 se publica ASP (Active Server Pages) como parte de la versión 3.0 de IIS. **ASP se creó buscando combinar HTML, scripts, componentes de ActiveX y Visual Basic, todo ello ejecutado en servidor para generar páginas web dinámicas**. ASP se basó en el modelo clásico de programación de aplicaciones basadas en el servidor, en el que los clientes enviaban solicitudes al servidor, que procesaba esas solicitudes y devolvía las respuestas correspondientes.

ASP utiliza ficheros con extensión “.asp” que llevan HTML y scripts (por defecto en Visual Basic) incrustados en etiquetas <% %> que ejecutan el código en el servidor. También permite añadir otros lenguajes con la expresión <%@ language=”<<lang>>” %>.

En el año 1997 ya había quejas de los desarrolladores pidiendo mecanismos para escribir código más limpio y poder separar la presentación, el HTML y los componentes de ActiveX, por lo que se inició un rediseño completo de ASP, pasando de un sistema de scripting integrado a un framework de desarrollo completo de aplicaciones web. Este nuevo framework se lanzaría con la nueva plataforma que Microsoft estaba desarrollando en ese momento, .NET Framework, y se llamaría ASP.NET.

13.2 ASP.NET

En 2002 se publica ASP.NET 1.0, como una evolución totalmente distinta de ASP, pero con el mismo objetivo: crear páginas web dinámicas dentro de un framework de desarrollo. A medida que se ha actualizado .NET Framework, se ha potenciado el desarrollo de ASP.NET y se han creado librerías como WCF o ADO.NET para añadir más funcionalidad a las aplicaciones web. Actualmente la última versión de ASP.NET es la versión 4.8 que coincide con la última versión de .NET Framework 4.8.

ASP.NET pasa a utilizar C# como su lenguaje preferente de desarrollo y dispone de tres modelos de programación que se detallan en los siguientes apartados: WebForms, MVC y WebPages (Razor).

En ASP.NET tanto aplicaciones como servicios web se despliegan en el servidor de aplicaciones IIS y deben llevar un fichero web.config. Este es un fichero de configuración XML en el que se definen:

- Dependencias
- Variables de configuración
- Configuración de la sesión, variables y persistencia (InProc, StateServer, SQLServer, Custom)
- Sistema de autenticación
- Cadenas de conexión

El otro elemento común en los proyectos de ASP.NET es la estructura de carpetas, en la cual se han definido una serie de elementos clave

- **App_Start**: Configuración inicial de aplicación
- App_Code: Módulos compilados en tiempo de ejecución
- **App_Data**: Datos de aplicación, cadenas de conexión o ficheros de datos
- App_LocalResources/App_GlobalResources: Ficheros de recursos
- **Bin**: Contiene código compilado (generalmente archivos .dll)
- **Scripts**: ficheros de script de navegador como JS
- **Content**: Hojas de estilo CSS y recursos

13.3 ASP.NET WEB FORMS

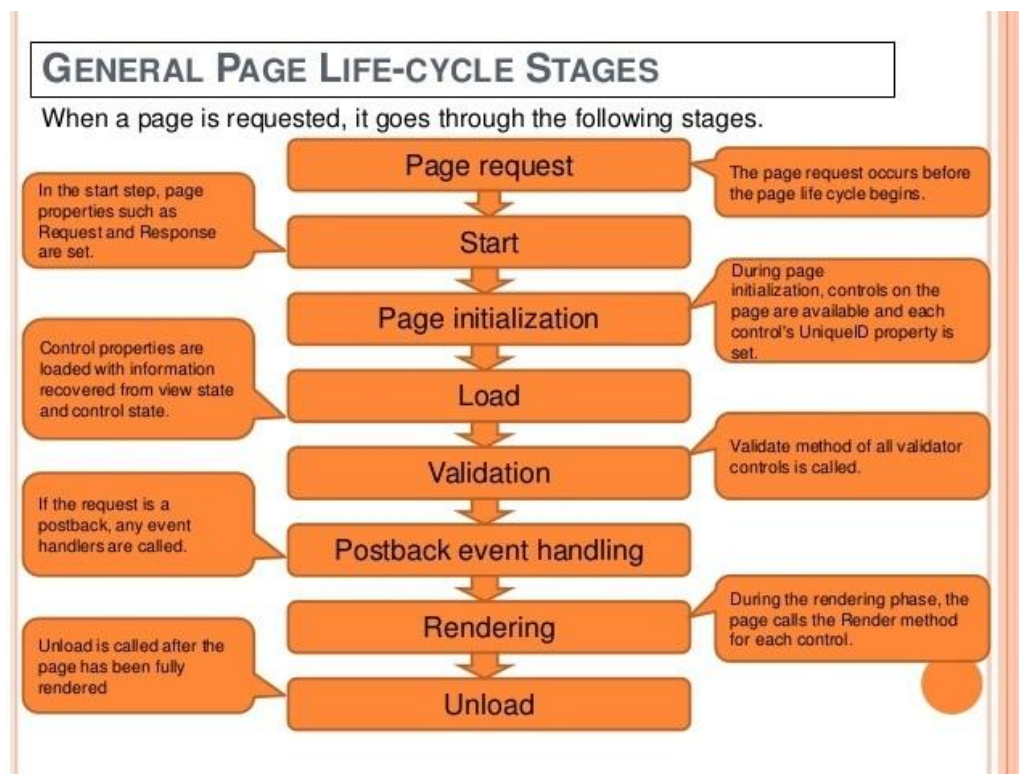
ASP.NET WebForms es el primero de los modelos de programación de ASP.NET y se centra en la creación de páginas XHTML con controles de servidor, que son el elemento central de WebForms, actuando como componentes reutilizables de generación de HTML y con capacidad de responder a eventos. En WebForms existen tres tipos de instrucciones principales usadas para el diseño de las páginas:

- Instrucciones asp
 - `<asp:textbox id='mytextbox' runat='server'>`
 - La instrucción anterior renderiza un campo de texto
- Código dinámicas
 - `<% -- dynamic code -- %>`
- Directivas
 - Instrucciones de cómo se debe procesar el fichero
 - `<%@ Page %>`

WebForms utiliza principalmente archivos “.aspx” que contienen la página XHTML con los controles de servidor, aunque existen otros relacionados como “.ascx” para controles de usuario.

Con la llegada de ASP.NET 2.0 se mejora el sistema de sesiones y de gestión de estado (con view state, application state y sesión state), mejoras en el sistema de cacheo y compilación dinámica y nuevas librerías disponibles. El otro gran cambio es el sistema Code-behind, por el cual cada página tendrá asociado un fichero de igual nombre y extensión “.aspx.cs” o “.aspx.vb” con el código dinámico de la página y el controlador. Este fichero permite generar una arquitectura más limpia (separación de modelo y lógica de presentación) y autogenerar código con la ayuda de IDEs como Visual Studio.

Todas las páginas disponen además de un ciclo de vida con varios eventos en los que se puede añadir código para modificar el comportamiento de la página y realizar acciones previas al envío al navegador web, como comprobaciones de autorización o inspección de ViewState.



<https://www.quora.com/What-is-page-life-cycle-of-asp-net>

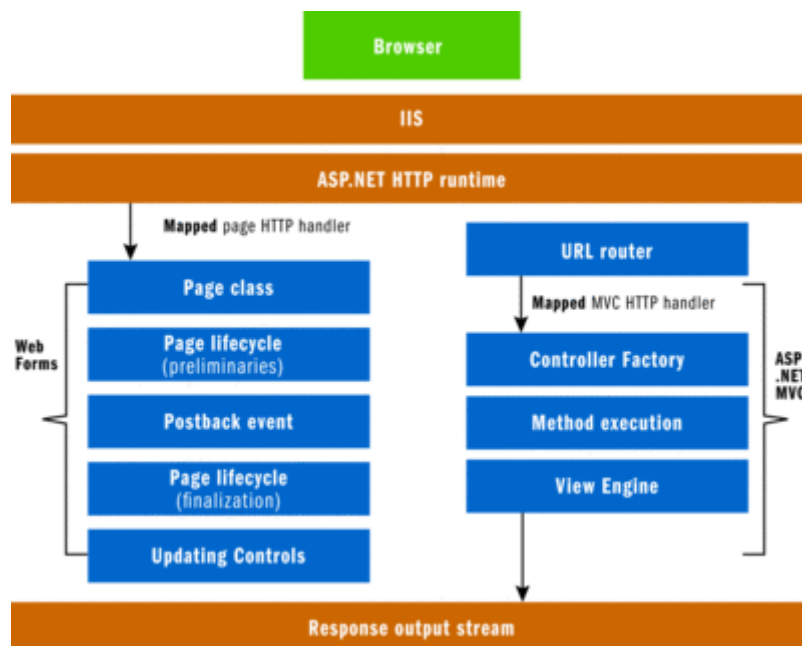
13.4 ASP.NET MVC

En 2007 con la llegada de .NET Framework 3.5, se publica un nuevo modelo de programación web llamado ASP.NET MVC. Este modelo se centra en el patrón MVC, la separación de responsabilidad, uso de TDD y los principios SOLID y la inversión de control con inyecciones de dependencias.

Inicialmente ASP.NET MVC utilizaba archivos “.aspx” al igual que WebForms, pero con la llegada de las plantillas Razor, se recomienda el uso de archivos “.cshtml” o “.vbhtml” a partir de ASP.NET MVC 3. Estos archivos incluyen directivas Razor y HTML. Estos archivos se almacenan por defecto en la carpeta “Views” junto con archivos de plantilla y componentes reutilizables también en formato “.cshtml”. Actualmente la última versión del framework es ASP.NET MVC 5.2.9.

ASP.NET MVC incluye mejoras de código asíncrono, gestión de estado de vista y sesión, contenedor de dependencias, validaciones en cliente y mejor integración con scripts JavaScript, así como uso de AJAX (publicado en 2007).

Pero el cambio más importante es el modelo MVC de desarrollo. En este modelo existe un fichero de código cuyo nombre acaba en “Controller” que serán los ficheros de controlador, al contrario que con WebForms, no se sigue un ciclo de eventos de renderizado si no que existen unos métodos de controlador y un motor de renderizado de páginas. Esta separación de controlador, vista y lógica permite un diseño más limpio, que facilita las pruebas unitarias y permite definir unas rutas más limpias.



<https://learn.microsoft.com/es-es/archive/msdn-magazine/2009/july/dino-esposito-on-comparing-web-forms-with-asp-net-mvc>

13.5 ASP.NET WEB PAGES Y RAZOR

Con la llegada de .NET Framework 4.0 en 2010 se busca sustituir el sistema de páginas “.aspx” por una versión más flexible y moderna que permita combinar código, componentes y scripts de cliente de forma sencilla y con una curva de aprendizaje baja. En 2011 se presenta **la sintaxis Razor y los ficheros “.cshtml”, “.vbhtml” o “.razor” junto con el modelo de desarrollo de ASP.NET WebPages. Este modelo de páginas permite desarrollar componentes reutilizables sin necesidad de escribir la estructura HTML completa y definir plantillas**

Razor se aplica también al sistema ASP.NET MVC 3 y se basa en un **fichero HTML en el que se incluyen directivas con el símbolo “@”**. No solo existe carga de variables o referencia a componentes, también se incluyen bloques de ejecución y de control como:

- @if/@switch/@for/@foreach
- @* comentarios *@
- @{ bloque de ejecución }
- @variables/ @@variable escapada

ASP.NET WebPages propone un nuevo modelo de desarrollo basado en páginas de tipo (MVVM – Model, View, ViewModel) que se puede combinar con ASP.NET MVC. Además de los ficheros “.cshtml” se crea un fichero de código asociado que actuará de ViewModel y genera una ruta a la página sin necesidad de controlador. Se utiliza una nueva carpeta interna “Pages” separada de las “Views” tradicionales de MVC de forma que se pueden desarrollar paginas aisladas, centradas en el negocio de forma más sencilla.

En 2012 y con llegada de .NET Framework 4.5 se añade un componente nuevo a la plataforma ASP.NET, la Web API. **Web API permite desarrollar servicios RESTful de forma sencilla sin necesidad de incluir todas las dependencias de MVC**, generando un servicio más rápido, compacto y versátil.

13.6 ASP.NET CORE

El campo a .NET Core supuso una reescritura del framework tradicional de ASP.NET que ahora permite el despliegue de servicios y aplicaciones web en múltiples entornos. No existen muchos cambios respecto al proceso de desarrollo ASP.NET MVC/WebPages manteniéndose las plantillas Razor. La última versión LTS es ASP.NET Core 6 y las diferencias más significativas respecto a ASP.NET MVC/WebPages son:

- Soporte de múltiples sistemas operativos y arquitecturas
- Simplificación de la estructura de carpetas (Las carpetas App_* se sustituyen por wwwroot y ficheros de configuración)
- Web.config desaparece de la estructura por defecto (salvo que se despliegue en IIS) en favor de appsettings.json. Este es un fichero de texto plano con formato JSON5 en el que se pueden definir variables de la aplicación. La definición de dependencias pasa a estar integrado en el proyecto
- Mejoras en la modularidad, inyección de dependencias, configuración inicial y rendimiento
- Cambios en APIs antiguas, pero manteniendo similitudes con MVC
- Uso intensivo de sistemas middleware independientes
- WebAPI 2 muy compacta y especializada en servicios RESTful, sin necesidad de importar un renderizador de HTML.

13.7 BLAZOR

Blazor es la nueva variante de ASP.NET Core que permite crear aplicaciones web interactivas utilizando C# y .NET en lugar de JavaScript. Blazor utiliza WebAssembly para compilar los ensamblados de la aplicación y ejecutarlos en el navegador, por lo que con el motor de plantillas Razor y enlaces dinámicos se crea un modelo de desarrollo MVU sin usar JavaScript. Esta tecnología todavía no tiene gran adopción.

Blazor se lanzó por primera vez en septiembre de 2018 con .NET Core 3.0, cuando se lanzó Blazor Server. Blazor Server es una versión de Blazor que utiliza una conexión de red en tiempo real basada en SignalR para enviar y recibir actualizaciones de la interfaz de usuario, lo que permite que las aplicaciones Blazor se ejecuten en el servidor y se rendericen en el cliente.

En mayo de 2020 con .NET 5, se lanzó Blazor WebAssembly, que permite a las aplicaciones Blazor ejecutarse directamente en el navegador web sin necesidad de una conexión en tiempo real con el servidor. El servidor envía los ficheros “.dll” al navegador web y son ejecutados por una implementación JavaScript de .NET que utiliza WebAssembly. Esta forma de trabajo es bastante controvertida por los problemas que genera enviar ficheros “.dll” a los navegadores, especialmente con sistemas WAF.

En noviembre de 2021 con .NET 6 se publica **Blazor Hybrid junto con MAUI para el desarrollo de aplicaciones híbridas para escritorio, web y móvil con una misma base de código en .NET.**

13.8 OTRAS TECNOLOGÍAS

Otras tecnologías de desarrollo web utilizadas en .NET son

- **SignalR:** Permite crear un servidor con conexión en tiempo real basado en WebSockets y Server-sent-events. Muy utilizado en sistemas chat, datos en vivo y servicios online como videojuegos. Disponible para .NET Framework (en su versión antigua) y .NET (Core)
- **Silverlight:** fue una tecnología desarrollada por Microsoft que permitía a los desarrolladores crear aplicaciones web ricas y multimediales utilizando el lenguaje de programación C# y .NET en lugar de JavaScript. Fue lanzado por primera vez en 2007 y se promovió como una alternativa a Flash.

14. DESARROLLO DE SERVICIOS – WCF

La limitación que presentan los servicios Web al utilizar solamente el protocolo HTTP es superada en .NET por los servicios WCF (Windows Communication Foundation), que pueden ejecutarse sobre TCP, HTTP, IPC o MSMQ. **WCF es un framework para construir aplicaciones orientadas a servicios.** Mediante WCF se pueden enviar datos como mensajes asíncronos entre los endpoints de dos servicios, que pueden estar hospedados en IIS o en otro servidor.

Las tecnologías de WCF están desarrolladas para .NET Framework y no son compatibles con .NET (Core). Con la llegada de .NET Core 3.1 se desarrolló una nueva librería inspirada en WCF llamada WCFCore solo disponible para entornos .NET (Core), más ligera y pequeña pero multiplataforma.

En el caso de los servicios SOAP, los mensajes intercambiados pueden tener el formato XML sin estar ensobrados en SOAP, el formato XML ATOM o un formato no XML como JSON. En WCFCore la funcionalidad de conexión SOAP se limita al estándar SOAP 1.2, por lo que algunas extensiones que no pertenecen al estándar no están soportadas como si ocurre en WCF.

15. ACCESO A DATOS – ADO.NET, LINQ Y ENTITY FRAMEWORK

.NET dispone de librerías y especificaciones propias para el acceso a BD desde múltiples proveedores como SQLServer, PostgreSQL, SQLite... e incluso BD NoSQL.

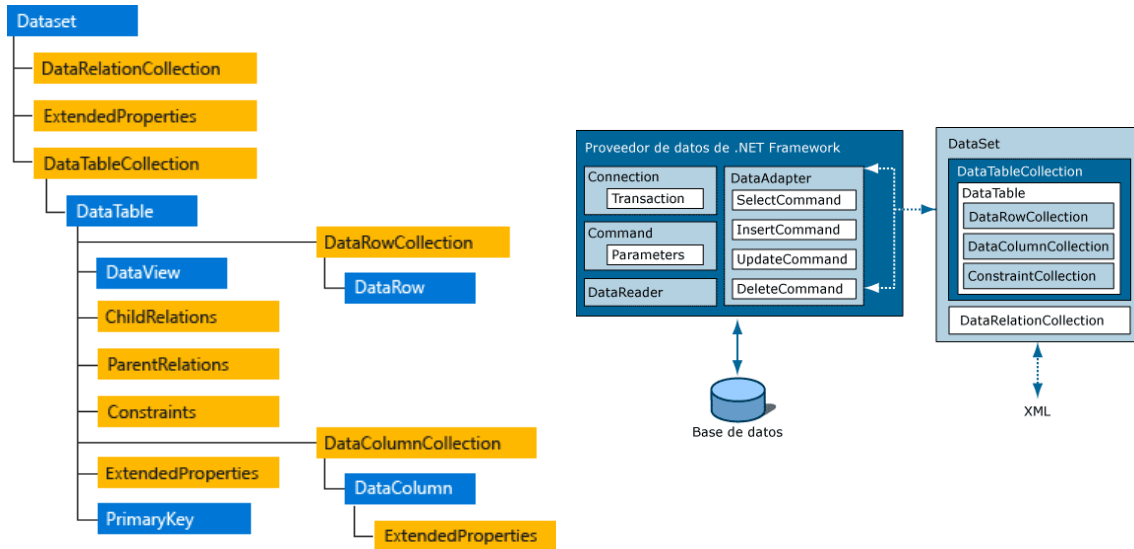
15.1 ADO.NET

ADO.NET es una tecnología de acceso a datos de Microsoft disponible en .NET Framework y .NET (Core) que proporciona comunicación principalmente entre sistemas relacionales (también algunos conjuntos no relacionales) a través de un conjunto común de componentes. Es una de las piezas fundamentales de .NET desde la versión .NET Framework 1.0. ADO.NET se podría considerar una evolución de la tecnología ActiveX Data Objects (ADO), pero se modificó tanto que es un producto totalmente nuevo.

ADO.NET consta de dos partes principales:

- **Data Providers:** Las clases que proporcionan el acceso a la fuente de datos, como por ejemplo una BD relacional como SQLServer. Dispone de 5 elementos clave: Connection, Command, Parameter, DataAdapter y DataReader.

- **DataSets y su jerarquía:** es un objeto en memoria que se utiliza para contener y manipular datos de una base de datos relacional. Es una representación independiente de la base de datos que permite a los desarrolladores trabajar con datos en un entorno desconectado.



<https://learn.microsoft.com/es-es/dotnet/framework/data/adonet/ado-net-datasets>

<https://www.ingenieriasystems.com/2014/04/revision-de-ado-net.html>

ADO.NET se adaptó a .NET Core y se mantiene gran parte de su funcionalidad. Recientemente se ha realizado un cambio a nivel interno para soportar nuevos sistemas de almacenamiento más modernos por el cual se cambia el ensamblado de acceso de System.Data a Microsoft.Data.

Ampliar información con el fichero “XVII_Accediendo a los Datos con ADO.NET” en la carpeta de “Doc adicional”.

15.2 LINQ

LINQ (Language Integrated Query) es una tecnología de Microsoft que permite hacer consultas en diversas fuentes de datos, como bases de datos, colecciones de objetos en memoria, XML y servicios web, **utilizando una sintaxis unificada y familiar**. Forma parte de .NET Framework 3.5 y esta presente en todo el ecosistema .NET desde entonces.

LINQ utiliza expresiones lambda para filtrar, ordenar y transformar los datos, lo que lo hace más fácil y eficiente de usar que otras tecnologías de consulta de datos. Algunos de los métodos más utilizados son: Select(), Where(), Join(), GroupBy(), Aggregate() o Sum() similares a SQL.

LINQ se integra con el lenguaje de programación C# y Visual Basic .NET, y proporciona una serie de métodos de extensión para los tipos de colecciones de .NET Framework, lo que permite a los desarrolladores escribir consultas en un estilo más fluido y fácil de leer.

Existen varios proveedores que implementan sus propias consultas como LINQ to SQL (utilizada en los inicios de ADO.NET), Entity Framework, Windows Search o XML entre otros.

Desde .NET Framework 4.0 se añade ParallelLINQ, una nueva instrucción que permite realizar consultas en paralelo, utilizando las excelentes capacidades multihilo de la plataforma .NET.

15.3 ENTITY FRAMEWORK

Entity Framework es un conjunto de tecnologías de Microsoft para el acceso y gestión de datos en aplicaciones .NET presente en la plataforma desde .NET Framework 3.5. Es un ORM (Mapeo de Objetos Relacionales) que permite a los desarrolladores trabajar con datos en forma de objetos y propiedades en lugar de lidiar directamente con sentencias SQL y estructuras de bases de datos.

Con Entity Framework, los desarrolladores pueden definir modelos de datos como clases y propiedades en un lenguaje de programación .NET, y luego interactuar con estos modelos como si fueran objetos en el código. Entity Framework se encarga de traducir las operaciones del objeto en consultas SQL para interactuar con la base de datos subyacente. Puede utilizar los proveedores de ADO.NET para consultas a BD relacionales o conectores específicos para servicios como NoSQL (CosmosDB).

Actualmente existen dos versiones de Entity Framework, la primera es Entity Framework 6, disponible en .NET Framework exclusivamente y es la última versión del framework original basado en ADO.NET que dispone de más funcionalidad y herramientas gráficas para la gestión de entidades y generación de código. La versión de .NET (Core) se llama Entity Framework Core y es un rediseño para adaptarse al sistema multiplataforma y a nuevos proveedores que se añaden progresivamente. Esta versión no tiene las mismas capacidades que la versión de .NET Framework, pero se añade funcionalidad con cada versión de .NET (actualmente en su versión 7). Dispone además de herramientas en el CLI de dotnet para la generación de código, migraciones y actualizaciones.

16. DESARROLLO DE APLICACIONES DE ESCRITORIO – WINFORMS Y WPF

WinForms y WPF son dos frameworks para el desarrollo de aplicaciones de escritorio en Windows.

WinForms (Windows Forms) es un framework de desarrollo gráfico que ha estado disponible desde .NET Framework 1.0. WinForms es un framework basado en ventanas y controles, que proporciona una amplia variedad de controles visuales, como botones, cuadros de texto, listas, etc. y eventos para manejar la interacción del usuario. WinForms es relativamente fácil de aprender y utilizar, y es compatible con versiones antiguas de .NET Framework.

WPF (Windows Presentation Foundation) es un framework que se introdujo en .NET Framework 3.0 y es la evolución natural de WinForms. WPF es una plataforma más moderna y avanzada para crear aplicaciones de escritorio en Windows, que se basa en la programación de interfaces gráficas de usuario (GUI) utilizando XAML (un lenguaje de marcado similar a HTML). WPF proporciona muchas características nuevas, como la capacidad de crear interfaces de usuario altamente personalizadas y animaciones fluidas, así como soporte para gráficos 2D y 3D. WPF también es compatible con otras tecnologías de Microsoft, como Silverlight y Windows Phone.

Desde .NET Core 3.0 están presentes en el entorno .NET (Core) gracias a rediseño de ambos frameworks que introducen cambios y mejoras sustanciales. Con .NET 6, se han introducido varias mejoras y nuevas características para WinForms y WPF. Por ejemplo, WinForms ahora admite la aceleración de hardware para mejorar el rendimiento, y WPF ahora tiene soporte para .NET MAUI (Multi-platform App UI), lo que permite a los desarrolladores crear aplicaciones de escritorio para múltiples plataformas.

Además, .NET 6 ha mejorado la compatibilidad entre los diferentes frameworks de Windows UI, lo que permite a los desarrolladores crear aplicaciones que combinan elementos de WinForms, WPF y otras tecnologías de Windows UI.

.NET (Core) incluye también la plataforma UWP para el desarrollo en Windows 8 y superior.

17. DESARROLLO DE APLICACIONES MULTIPLATAFORMA – XAMARIN, MAUI Y BLAZOR

WinForms y WPF nos permiten diseñar aplicaciones de escritorio para Windows, en el caso de desarrollo de aplicaciones multiplataforma y móviles disponemos de Xamarin, MAUI y Blazor. Estos frameworks (excepto Xamarin que existe desde .NET Core 1.0) necesitan .NET 6 o superior

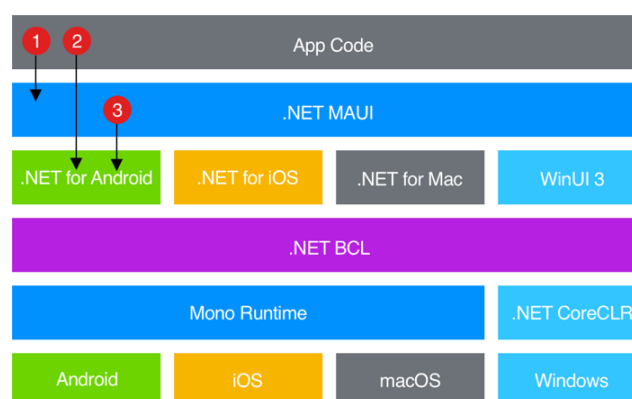
Xamarin es una plataforma de desarrollo de aplicaciones móviles que permite a los desarrolladores crear aplicaciones nativas para Android, iOS y macOS utilizando el ecosistema .NET. Xamarin fue adquirido por Microsoft en 2016 y ahora es parte de la familia de productos de Microsoft Visual Studio.

Xamarin permite a los desarrolladores crear aplicaciones nativas para múltiples plataformas utilizando un solo conjunto de herramientas y lenguaje de programación. Al utilizar C# y el framework .NET, los desarrolladores pueden compartir gran parte del código entre las diferentes plataformas, lo que ahorra tiempo y esfuerzo. Además, Xamarin proporciona acceso completo a las API nativas de cada plataforma, lo que permite a los desarrolladores crear aplicaciones móviles que se integran perfectamente con el sistema operativo subyacente.

Xamarin.Forms es una biblioteca de interfaz de usuario (UI) de código abierto para crear aplicaciones móviles híbridas para Android, iOS y Windows utilizando el framework .NET y el lenguaje de programación C#. Xamarin.Forms permite a los desarrolladores crear interfaces de usuario compartidas para múltiples plataformas con una única base de código.

.NET MAUI (Multi-platform App UI) es un framework de desarrollo de aplicaciones móviles multiplataforma de código abierto que permite a los desarrolladores crear aplicaciones nativas para iOS, Android, Windows y macOS utilizando un único conjunto de herramientas y código compartido.

.NET MAUI es una evolución de Xamarin.Forms y se basa en .NET 6. Proporciona una amplia variedad de características y herramientas para crear aplicaciones móviles, como controles de interfaz de usuario (UI). Está integrado dentro de .NET y reemplaza a Xamarin para el desarrollo de aplicaciones híbridas.



<https://learn.microsoft.com/es-es/dotnet/maui/what-is-maui?view=net-maui-7.0>

Finalmente, Blazor dispone desde .NET 6 de Blazor Hybrid que es un método de despliegue por el cual se puede desarrollar una aplicación ASP.NET que se adapta a entornos de escritorio y móviles. Además, Blazor Hybrid dispone de herramientas para desarrollar componentes en WinForms Blazor y WPF Blazor.

18. OTRAS TECNOLOGÍAS DE .NET Y LIBRERÍAS POPULARES

Otras tecnologías relevantes en el ecosistema .NET son:

- **WINAPI:** (Windows API) en C++, que permite un acceso a bajo nivel del sistema, por ejemplo, en lo referente a la gestión de memoria y el acceso al hardware.
- **ML.NET:** es un framework de aprendizaje automático de código abierto de .NET (Core). ML.NET proporciona una amplia variedad de algoritmos de aprendizaje automático para tareas comunes, como clasificación, regresión, agrupación y detección de anomalías. Se puede integrar con otras herramientas como Azure o PowerBI y utilizar modelos complejos de herramientas como Tensorflow.
- **Unity:** Un motor gráfico para el diseño de videojuegos, aplicaciones gráficas y de VR/AR construido sobre .NET (Core). Utiliza el compilador IL2CPP para transformar código C# a C++ nativo.

El ecosistema .NET dispone además de una gran comunidad de código abierto en el que destacan proyectos como:

- Desarrollo CQRS: MediatR, AutoMapper, Autofac
- Logging: Serilog, Log4Net
- Diseño distribuido: Polly,
- API: Newtonsoft.JSON, Swashbuckle Swagger, Fluent Validation, protobuf
- Testing: xUnit, NUnit, FluentAssertions